

Nichtlineare Optimierung

Operations Research SoSe 2026

Christian Grimme

07. Juli - 14. Juli 2026

Universität Münster, Wirtschaftsinformatik
Computational Social Science & Systems Analysis

Problemdefinition

Visualisierung

Analytische Lösungen

Deterministische numerische Verfahren

 Eindimensionale nichtlineare Optimierung

 Mehrdimensionale nichtlineare Optimierung

Wrap-Up

Motivation

- Innere Zusammenhänge realer Systeme lassen sich selten über lineare Modelle exakt beschreiben.
- Lineare Modelle:
 - unbestreitbar einfach
 - oft schnell und präzise lösbar (z.B. Simplex-Algorithmus)
- Beispiele in denen Nicht-Linearität (oft) unausweichlich ist:
 - Anfangsbeispiel der VL: Verschnittproblem
 - Kosten- und Effizienzprobleme (z.B. Projektteams in der Softwareentwicklung)
 - Modellierung physikalischer und technischer Systeme
 - ...

Problemdefinition

Nicht-lineares Optimierungsproblem

Definition

Ein nicht-lineares Optimierungsproblem ist gegeben durch

$$f(x_1, \dots, x_n) \stackrel{!}{=} \text{opt}$$

- n Entscheidungsvariablen $x_1, \dots, x_n, x_i \in \mathbb{R}$ und $n \in \mathbb{N}$ liegen in f in einem nicht-linearen Zusammenhang vor,
- (nicht-)lineare Ungleichheits-Nebenbedingungen
 $g_i(x_1, \dots, x_n) \leq b_i, i = 1, \dots, m$
- (nicht-)lineare Gleichheits-Nebenbedingungen
 $h_j(x_1, \dots, x_n) = b_j, j = 1, \dots, l$

Beispiele

Wir wollen den Materialverbrauch bei der Herstellung einer Dose minimieren. Funktionaler Zusammenhang auf Radius beschränkt:

$$f(r) = 8r^2 + \frac{2}{r}$$

- Randbedingungen? $\Rightarrow r > 0$: alles andere wäre unrealistisch

Anderes (höherdimensionales) Problem:

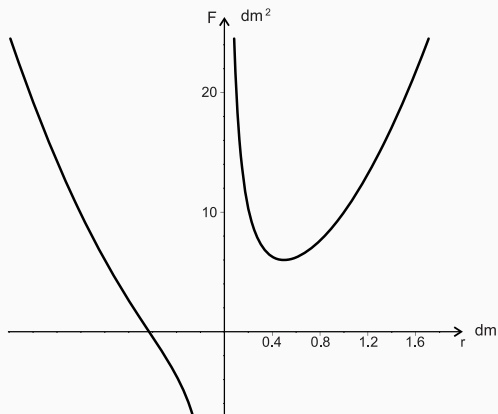
$$f(x_1, x_2) = 5x_1^2 - 3x_1x_2 + 7x_2^2 - 3x_2.$$

- keine Randbedingungen.

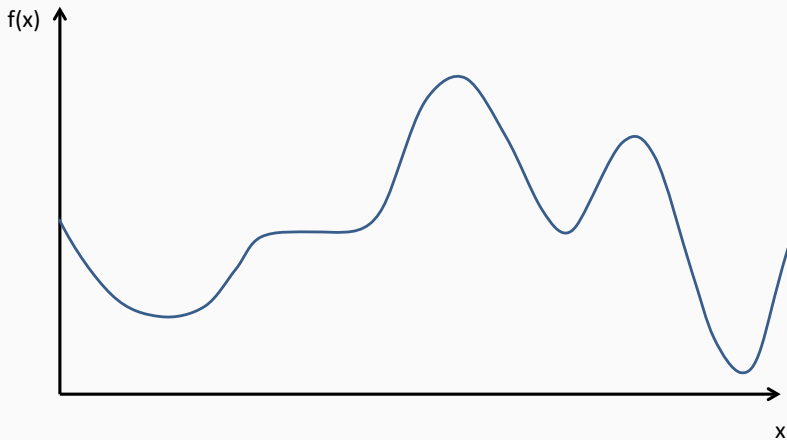
Visualisierung

Grafik zum Verschnittproblem

$$F(r) = 8r^2 + \frac{2}{r}.$$

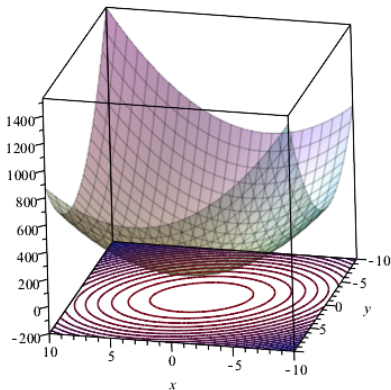


Eindimensionale Probleme

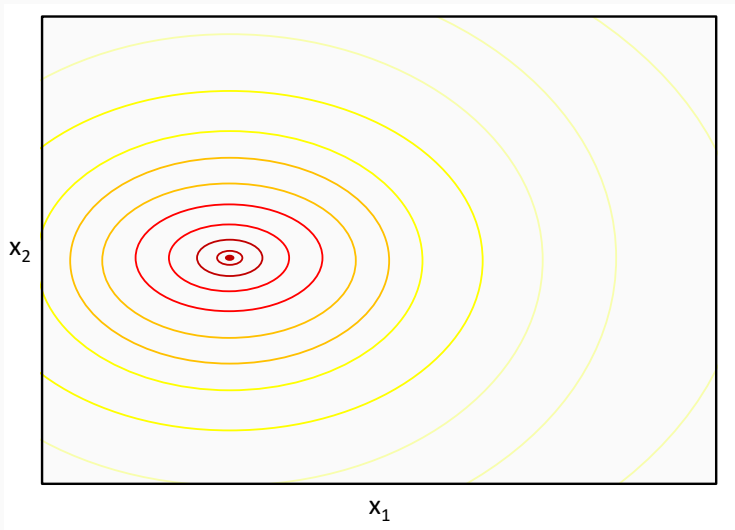


Beispiel: Mehrdimensionales Problem

$$f(x_1, x_2) = 5x_1^2 - 3x_1x_2 + 7x_2^2 - 3x_2.$$



Mehrdimensionale Probleme



Zwei Lösungsparadigmen

Direkte Verfahren:

- Formulierung über Parameter, jedoch ist eine geschlossene mathematische Form nicht zwingend nötig.
- Schrittweises Vorgehen auf den betrachteten Parametern um den Ziel(funktions)wert zu verbessern
- Trial and Error (Versuch und Irrtum)

Indirekte Verfahren:

- Mathematisch vollständige Formulierung wird vorausgesetzt
- Man versucht, mit analytischen Verfahren *in einem Schritt* zur Lösung zu gelangen
- Während des Schritts wird kein Test der Güte durchgeführt.

Analytische Lösungen

Zwei Arten von nicht-linearen Problemstellungen:

- Probleme ohne Restriktionen: keine Randbedingungen
- Probleme mit Restriktionen: Nebenbedingungen als Gleichungen und Ungleichungen

Weitere Unterscheidung:

- eine Veränderliche / Parameter $\Rightarrow$ Kurvendiskussion
- mehrere Veränderliche / Parameter $\Rightarrow$ Lagange-Methode

Grundsätzliches Vorgehen der Optimierung

Für eine gegebene Zielfunktion wird allgemein folgendermaßen vorgegangen:

1. Ermittle sogenannte *kritische Stellen*, also solche, die als Optimum in Frage kommen, und damit eine notwendige Bedingung erfüllen.
2. *Überprüfe* alle kritischen Stellen *auf Optimalität*. Dies wird als hinreichende Bedingung bezeichnet.

Eine kritische Stelle, die

- die notwendige und
- die hinreichende Bedingung erfüllt ist

lokal optimal.

Globales Optimum: *Randwertvergleich*, also ein Vergleich mit den Zielfunktionswerten am Definitionsbereichsrand (auch $-\infty$ und ∞).

Abschlussbetrachtung

- Mathematisch gut begründet und für geschlossene Formulierungen anwendbar.
- Aber: Zielfunktion muss vorhanden und Differenzierbarkeit muss gegeben sein.
- Exakte Methoden, die in einem Schritt zum (lokalen) Optimum führen.
- Aber:
 - Keine Algorithmen im eigentlichen Sinne, sondern Vorgehensmodelle.
 - Übertragbarkeit auf den Computer schwierig.
 - Ggf. sehr zeitaufwändig durchzuführen (z.B. alle Permutationen von aktiven und inaktiven Nebenbedingungen testen).

Deterministische Verfahren

Motivation: Lösen kontinuierlicher Problemstellungen mit dem Computer.

- Exakte numerische Verfahren: sind in der Lage, kontinuierliche Problemstellungen in endlicher Zeit mit unendlicher Genauigkeit zu lösen (z.B. Gauß-Algorithmus oder Simplex-Verfahren)
- Approximative numerische Verfahren: lösen eine Problemstellung näherungsweise und können nur bei unendlichem Zeiteinsatz auch unendliche Genauigkeit erreichen.

- deterministisch $\leftrightarrow$ randomisiert:
 - Deterministische Verfahren folgen einem festen Schema und landen, vom gleichen Anfangszustand gestartet, stets im gleichen Endzustand.
 - Randomisierte Verfahren integrieren Zufallseffekte oder basieren vollständig darauf. Ein fester Endzustand kann selbst bei gleichem Startzustand nicht garantiert werden.
- simultan $\leftrightarrow$ sequentiell:
 - Simultan arbeitende Verfahren überprüfen die Optimalität von Kandidatenlösungen nebenläufig.
 - Sequentielle Verfahren gehen bei der Suche nach einem Optimum schrittweise vor. Dabei können Informationen aus vorherigen Schritten integriert werden.

Gittersuche:

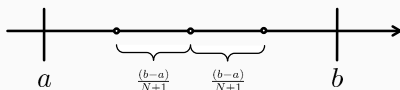
- Platziere Punkte auf Gitterknoten im Suchraum und untersuche sie unabhängig voneinander.
- Der beste Punkt wird als Optimum gewählt.

Betrachtung im Intervall $[a, b]$ (eindimensional):

- Verteile N Punkte gleichmäßig zwischen a und b .
- Bewerte alle N Punkte.

Wie gut ist das Verfahren?

Länge des Unsicherheitsbereichs für einen gewählten Punkt:



$$\ell_N = \frac{2}{N+1}(b-a).$$

Nötige Unterteilungen, um eine maximale Ungenauigkeit von ε sicherzustellen?

$\ell_n = \varepsilon$ und damit ergibt sich nach Umstellen:

$$N = \frac{2(b-a)}{\varepsilon} - 1$$

Beispiel: Simultane Suche

Intervall $[a, b] = [0, 1]$ und $\varepsilon = 0.01$:

Durch einfaches Einsetzen ergibt sich:

$$N > \frac{2}{\varepsilon} - 1 = \frac{2}{0.01} - 1 = 199,$$

Vorbereitungen: Boxing-In

Ziel: Auffinden eines geeigneten Startintervalls für simultan/sequentielle) Optimierung.

Festlegung: $x_{(0)}$ ist der Startpunkt, $f(x)$ eine gegebene Funktion.

1. Wähle feste Schrittweite s und bestimme $x_{(1)} = x_{(0)} + s$.
2. Wenn $f(x_{(1)}) < f(x_{(0)})$, dann fahre fort mit

$$x_{(k+1)} = x_{(k)} + s$$

bis gilt

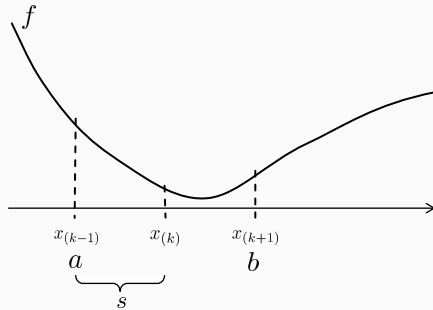
$$f(x_{(k+1)}) > f(x_{(k)}).$$

3. Das Intervall ergibt sich als $[a, b]$ mit $a = x_{(k-1)}$ und $b = x_{(k+1)}$.

Ist Bedingung (2) zu Beginn nicht erfüllt, führe Iteration in entgegengesetzter Richtung durch:

$$x_{(k+1)} = x_{(k)} - s.$$

Boxing-In graphisch



Beispiel: Boxing-In

Für die Funktion $f(x) = x^2$ wird der Startpunkt $x_{(0)} = -4$ gewählt. Die Schrittweite $s = 1\frac{1}{2}$

$$f(x_{(0)}) = 16.$$

k	$x_{(k)}$	$f(x_{(k)})$
0	-4	16
1	-2.5	6.25
2	-1	1
3	0.5	0.25
4	2	4

Intervall: $[-1, 2]$

Boxing-In, Erweiterungen

- Das Verfahren bestimmt ein Intervall mit Ungenauigkeit von $2 \cdot s$ um das Optimum.
- Laufzeit des Verfahrens abhängig von Startpunkt und Schrittweite (ggf. sogar unendliche Laufzeit)
- Schnelleres Vorgehen:
 - Iterationsformel: $x_{(k+1)} = x_{(0)} + 2^k s$
 - anwenden, bis $f(x_{(k+1)}) > f(x_{(k)})$
 - $b = x_{(k+1)}$
 - von b aus in entgegengesetzte Richtung a analog bestimmen.

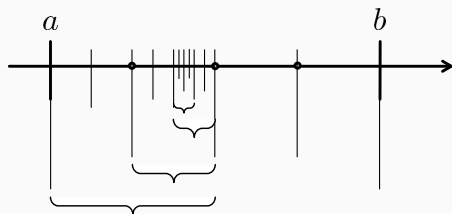
- Sachverhalte und Überlegungen sind in einer Dimension relativ einfach.
- Mehrdimensionale Optimierungsverfahren nutzen intern eindimensionale Strategien:
 - Einsatz als lokale Suchverfahren in komplexen Verfahren.
 - Daher ist es wichtig Eigenschaften und Einsetzbarkeit beurteilen zu können.
- Wir betrachten:
 - Intervallschachtelung
 - Interpolationsverfahren
- Natürlich gibt es auch noch unzählige andere Methodenvarianten und Verfeinerungen.

- Zentrale Herausforderung:
 - Erreichen einer guten Lösung mit wenigen Schritten, also große Genauigkeit bei wenig Laufzeit
- Randbedingung:
 - Wenige Funktionsauswertungen / Testpunkte, da jede Auswertung bzw. jeder Test sehr teuer sein kann.

Intervallschachtelung

- Motiviert aus der simultanen Suche, jedoch iterative Weiterführung
- Wiederholung der äquidistanten Unterteilung:
 - Bestimme das Teilintervall das den bisher besten Testpunkt enthält.
 - Lege N äquidistante Punkte in das Teilintervall.
 - Fahre analog fort bis eine festgelegte Ungenauigkeit von ε unterschritten wird.
- Die Ungenauigkeit nimmt exponentiell ab.

Schematisches Beispiel



Unsicherheitsbereich und Aufwand

Wählen wir jeweils drei gleich verteilte Testpunkte im Intervall, so ergibt sich für die Unsicherheitslänge folgende Entwicklung:

$$\begin{aligned}\ell_1 &= (b - a) \\ \ell_2 &= \frac{2}{3+1}(b - a) = \frac{1}{2}(b - a) \\ \ell_3 &= \frac{1}{2} \left(\frac{1}{2}(b - a) \right) \\ &\vdots \\ \ell_N &= \frac{1}{2^{N-1}}(b - a) = \varepsilon > 0\end{aligned}$$

Daraus folgt:

$$2^{N-1} = \left(\frac{b - a}{\varepsilon} \right) \Leftrightarrow N - 1 = \log_2 \left(\frac{b - a}{\varepsilon} \right) \Leftrightarrow N = 1 + \log_2 \left(\frac{b - a}{\varepsilon} \right)$$

Dichotome Suche

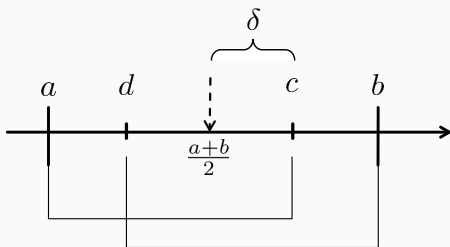
- Alternativer Ansatz zur Intervallschachtelung mit drei Testpunkten: es werden nur zwei Testpunkte benötigt.
- Im Intervall $[a, b]$ werden durch Testpunkte c und d zwei überlappende Intervalle $[a, c]$ und $[d, b]$ konstruiert:
 $a < d < c < b$.
- Die Überlappung ist durch δ von der Intervallmitte gegeben:

$$c = \frac{a + b}{2} + \delta$$

$$d = \frac{a + b}{2} - \delta$$

- Nächste Iteration:
 - $f(d) > f(c)$: Das nächste betrachtete Intervall ist $[d, b]$.
 - $f(d) \leq f(c)$: Das nächste betrachtete Intervall ist $[a, c]$

Schematisches Beispiel



Fibonacci-Suche

- Angelehnt an dichotome Suche
- δ wird theoretisch so gewählt, dass pro Iteration nur ein neuer Testpunkt benötigt wird.
- Statt δ direkt zu bestimmen wird eine Schrittweite s_k für Schritt k bestimmt.
- Grundlage des Verfahrens: Fibonacci-Zahlen
 - definiert als Fibonacci-Folge
 - Anfangsbedingung:

$$f_0 = f_1 = 1$$

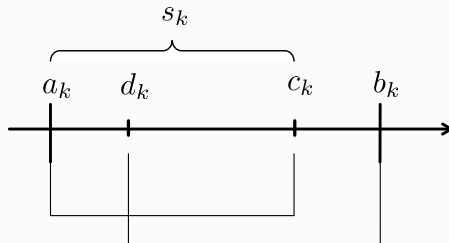
- Nächster Wert durch Addition der beiden direkten Vorgängerglieder:

$$f_k = f_{k-1} + f_{k-2}.$$

- Bekannte Folge:

$$1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, \dots$$

Schema der Methode (1)



Voraussetzung: es liegen N Fibonacci-Zahlen für N algorithmische Schritte vor, Intervall $[a_k, b_k]$ sei gegeben:

$$s_k = \frac{f_{N-(k+1)}}{f_{N-k}} (b_k - a_k)$$

$$c_k = a_k + s_k$$

$$d_k = b_k - s_k$$

Schema der Methode (2)

Auswahl des als nächstes betrachteten kleineren Teilintervalls (Minimierungsprobleme):

$$\cdot f(d_k) > f(c_k) :$$

$$a_{k+1} = d_k$$

$$b_{k+1} = b_k$$

$$\cdot f(d_k) \leq f(c_k) :$$

$$a_{k+1} = a_k$$

$$b_{k+1} = c_k$$

Eigenschaften der Methode

Wichtige Eigenschaft: Einer der verwendeten Suchpunkte kann wiederverwendet werden.

$$\cdot f(d_k) > f(c_k) :$$

$$d_{k+1} = c_k$$

$$\cdot f(d_k) \leq f(c_k) :$$

$$c_{k+1} = d_k$$

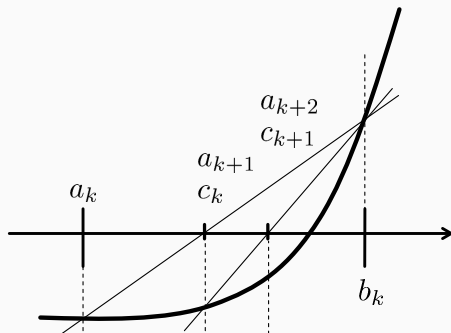
Nachteile:

- Anzahl der Suchschritte ist durch N vorgegeben
- N Fibonacci-Zahlen müssen erzeugt und gespeichert werden.

Regula Falsi

- Einfaches Interpolationsverfahren (lineares Modell)
- Bestimmen der Nullstelle der Ableitung
- Voraussetzung:
 - zwei Testpunkte a_k und b_k
 - die Steigung $f'(a_k)$ und $f'(b_k)$
 - Steigungen haben unterschiedliches Vorzeichen
- Ausnutzen des Zwischenwertsatzes: Es muss für die lineare Funktion einen Nulldurchgang geben
⇒ Kandidat für Extremstelle

Schematisches Beispiel



Der Graph ist ein Steigungsgraph (1. Ableitung)

Am angenäherten Punkt verrät die Steigung der Sekante, ob es sich um ein Minimum, Maximum oder Sattelpunkt handelt, zumindest annähernd.

Iterationsformel

Die Sekante ergibt sich über eine einfache lineare Interpolation der Art

$$f'(c_k) = f'(a_k) + \frac{f'(b_k) - f'(a_k)}{b_k - a_k}(c_k - a_k)$$

Wir suchen natürlich: $f'(c_k) = 0$

$$\begin{aligned}\frac{f'(b_k) - f'(a_k)}{b_k - a_k} c_k &= -f'(a_k) + \frac{f'(b_k) - f'(a_k)}{b_k - a_k} a_k \\ c_k &= a_k - f'(a_k) \frac{b_k - a_k}{f'(b_k) - f'(a_k)}.\end{aligned}$$

Bestimme a_{k+1} und b_{k+1} mittels c_k :

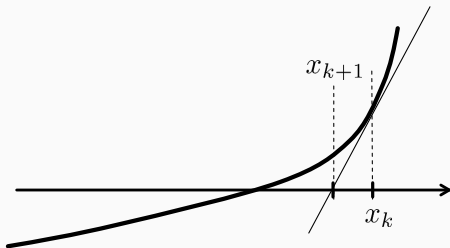
- $f'(c_k)$ und $f'(a_k)$ haben gleiches Vorzeichen:
 $\Rightarrow a_{k+1} = c_k$ und $b_{k+1} = b_k$.
- $f'(c_k)$ und $f'(b_k)$ haben gleiche Vorzeichen:
 $\Rightarrow a_{k+1} = a_k$ und $b_{k+1} = c_k$.

Eigenschaften der Methode

- Terminierung: Lösung exakt getroffen oder vorher festgelegte Genauigkeit erreicht: $f(c_k) < \varepsilon$
- Bestimmung der Art des Extremums: evtl. Steigung der letzten Sekante betrachten (keine Garantie).
- Die Steigung in den Testpunkten muss bestimmbar sein.

Newton-Raphson-Interpolation

- Erneut ein Interpolationsverfahren mit linearem Modell
- Betrachtung der Tangente in einem Punkt und der daraus resultierenden Tangentenfunktion (Gerade)
- Suche nach Nullstelle



Iterationsformel (1)

Wir betrachten zuerst das Vorgehen für eine Funktion $f(x)$:

Bekannte Größen: Suchpunkt x_k , Funktionswert $f(x_k)$, Steigung im Suchpunkt $f'(x_k)$

Ermittle mit x_{k+1} und den bekannten Größen die Tangentenfunktion:

$$t(x_{k+1}) = f(x_k) + f'(x_k)(x_{k+1} - x_k)$$

Sei x_{k+1} gerade die gesuchte Nullstelle, dann

$$t(x_{k+1}) = 0 = f(x_k) + f'(x_k)(x_{k+1} - x_k).$$

Daraus folgt:

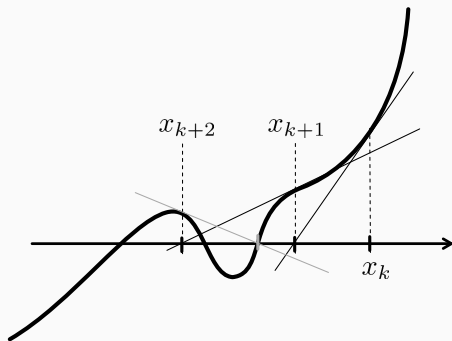
$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Iterationsformel (2)

Wir sind jedoch an der Nullstelle der ersten Ableitung interessiert:

$$x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}.$$

Schematisches Beispiel:



Eigenschaften der Methode (1)

Besonderheit: Quadratische Probleme

Für Probleme der Art $f(x) = ax^2 + bx + c$ ermittelt die Methode das Optimum in einem Schritt.

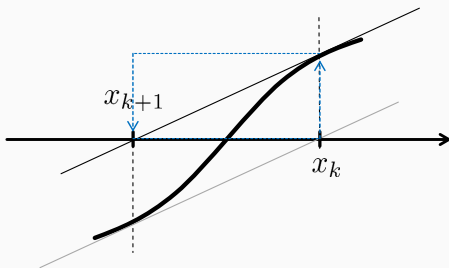
$$x_1 = x - \frac{2ax + b}{2a} = x - x - \frac{b}{2a} = -\frac{b}{2a}.$$

Ist aber auch klar, da die erste Ableitung einer quadratischen Funktion eine Gerade ist.

Eigenschaften der Methode (2)

Besonderheiten für Probleme größer als 2. Ordnung:

- Die Methode kann in einen stabilen Zyklus gelangen:



- Die Methode kann divergieren. (z.B. $f(x) = \sin(x)$)

Lagrange-Interpolation

- Ableitungsfreies Interpolationsverfahren
- Zumeist quadratisches Modell
- Es werden iterativ Ersatzpolynome für die betrachtete Funktion untersucht.
- Ersatzmodell entsteht aus Stützstellen und Funktionswerten (hier drei Stützstellen für quadratisches Polynom)

Allgemeine Polynome

Seien $n + 1$ Stützstellen gegeben. Erzeuge ein Polynom n -ten Grades aus $n + 1$ Lagrange-Grundpolynomen und $n + 1$ Funktionswerten:

Lagrangepolynom für Stützstelle i :

$$\begin{aligned} L_i(x) &= \prod_{k=0, k \neq i}^n \frac{x - x_k}{x_i - x_k} \\ &= \frac{x - x_0}{x_i - x_0} \cdot \frac{x - x_1}{x_i - x_1} \cdots \frac{x - x_{i-1}}{x_i - x_{i-1}} \cdot \frac{x - x_{i+1}}{x_i - x_{i+1}} \cdots \frac{x - x_n}{x_i - x_n} \end{aligned}$$

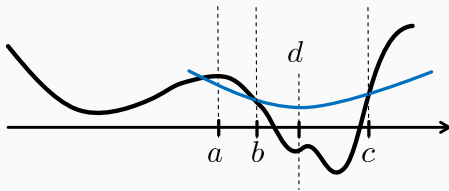
Polynom n -ten Grades:

$$p(x) = \sum_{i=0}^n f(x_i) L_i(x).$$

Quadratisches Polynom

- Für die konkrete Anwendung im Optimierungsfalle: Polynom vom Grad 2.
- Wir haben drei Stützpunkte a , b und c , also auch drei Lagrange-Grundpolynome.
- Es ergibt sich das Polynom:

$$p_2(x) = f(a) \frac{(x-b)(x-c)}{(a-b)(a-c)} + f(b) \frac{(x-a)(x-c)}{(b-a)(b-c)} + f(c) \frac{(x-a)(x-b)}{(c-a)(c-b)}$$



Iterationsformel (1)

Bestimme den Punkt, wo das Ersatzpolynom ein Minimum aufweist. Dies ist dort, wo die erste Ableitung 0 wird. Direkte Angabe des Punktes:

$$d = \frac{1}{2} \cdot \frac{(b^2 - c^2)f(a) + (c^2 - a^2)f(b) + (a^2 - b^2)f(c)}{(b - c)f(a) + (c - a)f(b) + (a - b)f(c)}$$

Fallunterscheidungen bzgl. d :

1. Fall:

$$\left. \begin{array}{l} a_{k+1} = a_k \\ b_{k+1} = d_k \\ c_{k+1} = b_k \end{array} \right\}, \text{ wenn } \left\{ \begin{array}{l} a_k < d_k < b_k \text{ und} \\ f(d_k) < f(b_k) \end{array} \right.$$

2. Fall:

$$\left. \begin{array}{l} a_{k+1} = d_k \\ b_{k+1} = b_k \\ c_{k+1} = c_k \end{array} \right\}, \text{ wenn } \left\{ \begin{array}{l} a_k < d_k < b_k \text{ und} \\ f(d_k) > f(b_k) \end{array} \right.$$

Iterationsformel (2)

3. Fall:

$$\left. \begin{array}{l} a_{k+1} = b_k \\ b_{k+1} = d_k \\ c_{k+1} = c_k \end{array} \right\}, \text{ wenn } \left\{ \begin{array}{l} b_k < d_k < c_k \text{ und} \\ f(d_k) < f(b_k) \end{array} \right.$$

4. Fall:

$$\left. \begin{array}{l} a_{k+1} = a_k \\ b_{k+1} = b_k \\ c_{k+1} = d_k \end{array} \right\}, \text{ wenn } \left\{ \begin{array}{l} b_k < d_k < c_k \text{ und} \\ f(d_k) > f(b_k) \end{array} \right.$$

Eigenschaften der Methode

- Das Verfahren terminiert, wenn eine gewünschte Genauigkeit erreicht wurde (Ausdehnung des Suchintervalls oder eine angestrebte Mindestqualität des Zielfunktionswertes) .
- Verfahren ist durch die quadratische Ersatzfunktion bestimmt. Es nimmt an, dass die Zielfunktion in der Nähe des Optimums eine quadratische Ausprägung aufweist und "gutmütig" ist.
- Für Probleme mit quadratischer Zielfunktion ermittelt das Verfahren die optimale Lösung in einem einzigen Iterationsschritt!

Mehrdimensionale Optimierung

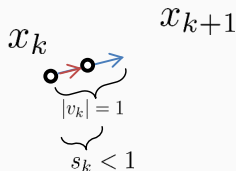
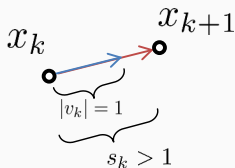
- Im mehrdimensionalen Suchraum ist der Aufwand für das Finden eines Optimums erheblich höher:
 - Gittersuche im Eindimensionalen: N Suchpunkte
 - Gittersuche im n -Dimensionalen: $N = \prod_{i=1}^n N_i$ Suchpunkte (N_i Untersuchungsichte in Dimension i)
- Exponentielle Zunahme der Testpunkte bei "Flutung" des Suchraums.
- "Fluch der Dimensionen"
- Iterative Strategien sind oft vorzuziehen: Wir schauen uns fünf Varianten an.

Allgemeine Iterationsformel

Im Mehrdimensionalen lautet die Iterationsformel:

$$x_{k+1} = x_k + s_k \cdot v_k$$

- $x_i \in \mathbb{R}^n$ ein Vektor im Suchraum
- $s_k \in \mathbb{R}$ eine skalare Schrittweite oder ein Schrittweitenvektor (dann natürlich $s_k \in \mathbb{R}$)
- $v_k \in \mathbb{R}^n$ ein Richtungsvektor mit Einheitslänge

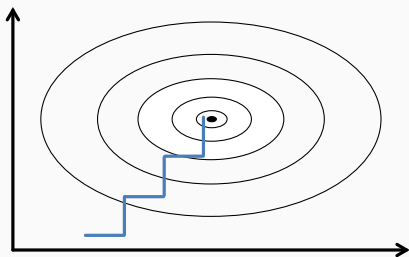


Einfachster aber modifizierbarer Algorithmus:

Eingabe: $x_0 \in \mathbb{R}^n$ als Startpunkt, n Suchraumdimension, s_0 initiale Schrittweite

- 1: Boxing-In in jeder Suchraumdimension.
- 2: **while** Terminierungskriterium nicht erfüllt **do**
- 3: **for** $i = 1, \dots, n$ **do**
- 4: Explorativer Schritt (ggf. lineare Suche) in Dimension i .
- 5: Überprüfe die Qualität des neuen Punktes und übernimm diesen ggf. als neuen Startpunkt.
- 6: Passe ggf. die Schrittweite an (abh. von (5)).
- 7: **end for**
- 8: **end while**

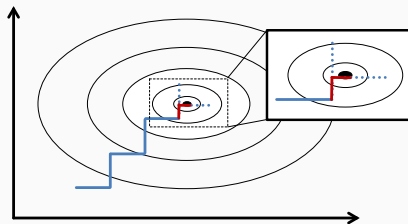
Variante: Konstante Schrittweite



Explorativer Schritt mit fester Schrittweite s in jeder Dimension:

- $x_{k+1} = x_k + s \cdot e_i$ oder
- $x_{k+1} = x_k - s \cdot e_i$

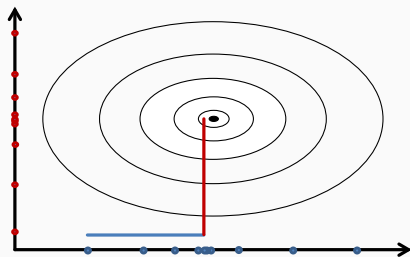
Variante: Schrittweitenanpassung



Explorativer Schritt mit fester Schrittweite s_i in jeder Dimension:

- Anpassung der Schrittweite (Vergrößerung) im Erfolgsfalle
- Anpassung der Schrittweite (Verkleinerung), wenn kein Fortschritt mehr erzielt wird.
- Anpassungsfaktor ist Gegenstand unzähliger Strategien.
- Normalerweise: $s_k \in \mathbb{R}^n$.

Variante: Lineare Suche



Explorativer Schritt mittels linearer Suche in jeder Dimension:

- Einsatz eindimensionaler Suchverfahren (Fibonacci-Suche, Goldener Schnitt, Lagrange-Interpolation)
- Exakte Anpassung der Schrittweite an lokale eindimensionale Gegebenheit (relatives Optimum).

Eigenschaften der Methode(n)

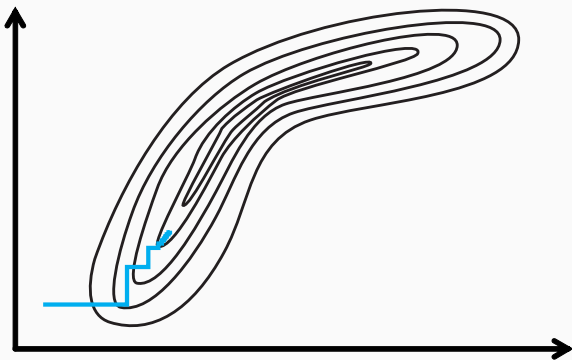
Vorteile:

- Einfach und direkt zu implementieren, daher häufig angewendet.
- Konvergiert (lokal) für exakte Schrittweitenanpassung und stetig differenzierbare Probleme

Nachteile:

- Konvergenzgeschwindigkeit ist stark von der Form der Funktion abhängig.
- Langsame Konvergenz, wenn Konturen nicht konzentrisch oder in Tälern.
- Diskontinuitäten können verfehlt/übersprungen werden.
- Feste Ausrichtung an Koordinatenachsen schränkt die Methode ein.

Beispiel: Langsame Konvergenz



Mustersuche (Hooke & Jeeves)

- Teilweises Aufheben der Bindung an die Koordinatenrichtungen.
- Erkennen von "Mustern" im Suchvorgehen.
- Zwei verschiedene Suchschritte:
 - Exploration: einfachster Ansatz der Koordinatensuche mit Schrittweitenhalbierung
 - Extrapolation: Resultierende Richtung der vorhergehenden Schritte als zusätzliche Suchrichtung berücksichtigen.
- Besonderheit: Extrapolationsschritte werden nicht sofort ausgewertet, sondern direkt ein Explorationsschritt angeschlossen.

Algorithmus von Hooke und Jeeves (1)

Exploration:

Eingabe: $x \in \mathbb{R}^n$ als Startpunkt, $q \in \mathbb{R}$ als Qualitätswert einer Vergleichslösung, $s \in \mathbb{R}^n$ Schrittweitenvektor

Ausgabe: $(x', f(x'))$

- 1: $x' = x; \quad f(x') = q$
- 2: **for** $i = 1, \dots, n$ **do**
- 3: **if** $f(x' + s_i \cdot e_i) < f(x')$ **then**
- 4: $x' := x' + s_i \cdot e_i; \quad f(x') = f(x' + s_i \cdot e_i)$
- 5: **else**
- 6: **if** $f(x' - s_i \cdot e_i) < f(x')$ **then**
- 7: $x' = x' - s_i \cdot e_i; \quad f(x') = f(x' - s_i \cdot e_i)$
- 8: **end if**
- 9: **end if**
- 10: **end for**

Algorithmus von Hooke und Jeeves (2)

Hauptschleife:

Eingabe: $x_0 \in \mathbb{R}^n$ als Startpunkt, n Suchraumdimension, $s \in \mathbb{R}^n$
initialer Schrittweitenvektor

- 1: $X = x_0$.
- 2: **while** Terminierungskriterium nicht erfüllt **do**
- 3: $(y, f(y)) := \text{Exploration}(x, f(x), s)$
- 4: **if** $y = x$ **then**
- 5: $s := \frac{1}{2}s$
- 6: **else**
- 7: Musterschritt(x, y, n, s)
- 8: **end if**
- 9: **end while**

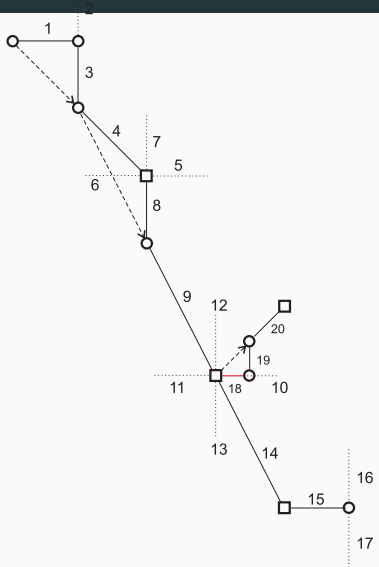
Algorithmus von Hooke und Jeeves (3)

Musterschritt:

Eingabe: $x, y \in \mathbb{R}^n$, n Suchraumdimension, $s \in \mathbb{R}^n$ Schrittweitenvektor

```
1: erfolg := true
2: while erfolg do
3:    $v := x + 2(y - x)$ 
4:    $(z, f(z)) = \text{Exploration}(v, f(y), s)$ 
5:   if  $f(z) < f(y)$  then
6:      $x := y; \quad y := z$ 
7:   else
8:     if  $f(v) < f(y)$  then
9:        $x := y; \quad y := v$ 
10:    else
11:      erfolg := false
12:       $x := y$ 
13:    end if
14:  end if
15: end while
```


Schematisches Beispiel



Eigenschaften der Methode

- Sehr erfolgreiches Verfahren.
- Der Musterschritt konstruiert sukzessive eine Näherung der Abstiegsrichtung.
- Kann aufgrund des Musterschrittes schnell Täler durchschreiten.
- Starke Krümmung von Tälern stellt jedoch ein Problem dar.
- Koordinatensuche bei Exploration findet ggf. keine Verbesserung bei Diskontinuitäten (vorzeitige Terminierung).

Methode des steilsten Abstiegs

- Bereits die vorherigen Methoden (Hooke & Jeeves, Nelder & Mead) haben versucht eine gute Abstiegsrichtung aus Explorationsinformationen zu generieren.
- Der direkte Weg ist die Nutzung der Gradienteninformation:
 - Gradient $\nabla f(x)$ zeigt in Richtung des größten Anstiegs
 - Verwende $-\nabla f(x)$ für Minimierungsprobleme

Vorgehen der Methode

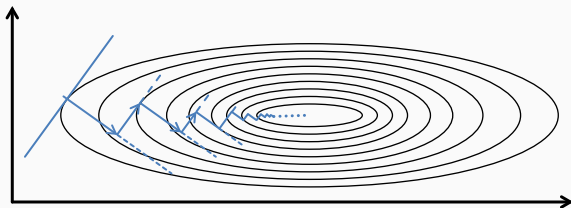
Sei $x_0 \in \mathbb{R}^n$ Startpunkt und an jeder Stelle der Gradient $\nabla f(x)$ bestimmbar:

1. Bestimmt für x_i den Gradienten $\nabla f(x_i)$ und verwende $v_i = -\nabla f(x_i)$ als Abstiegsrichtung.
2. Bilde $x' = x_i + s_i \cdot v_i$, wobei die Schrittweite s_i unterschiedliche bestimmt werden kann:
 - wähle s_i konstant (kleinschrittige Variante)
 - wähle s_i exakt durch eindimensionale Suche (langschrittige Variante)
3. Wenn eine Verbesserung erzielt wurde, $x_{i+1} := x'$

Dieses Vorgehen wird fortgesetzt, bis ein Terminierungskriterium erreicht ist, oder kein Fortschritt mehr erzielt wird (insbes. durch konstante Schrittweite).

Eigenschaften des Verfahrens

- Verfahren ist schnell für Funktionen mit symmetrischen Konturen (am liebsten quadratische Probleme)
- Ausgeprägte elliptische Formen können bereits ein Problem sein:
 - Zick-Zack-Verhalten
 - langsame Konvergenz
 - ähnliches Verhalten wie Koordinatensuche



Wrap-Up

Wir haben ...

- ... nicht-lineare Optimierungsprobleme kennengelernt.
- ... verschiedene numerische, eindimensionale Verfahren zur (iterativen und approximativen) Lösung kennengelernt.
- ... Vor- und Nachteile besprochen.
- ... mehrdimensionale Problemstellungen betrachtet und (approximativ) gelöst.

Fragen?